Análise e Desenvolvimento de Sistemas – 5° Período

Alunos:

Rayssa Leopoldina de Araujo R.A 24169768-2

AVALIAÇÃO BIMESTRAL

Implementação e Análise de Estruturas de Dados

1. INTRODUÇÃO

Este relatório apresenta a implementação prática e a análise de desempenho de diferentes estruturas de dados e algoritmos. O objetivo é consolidar os fundamentos teóricos através da experimentação empírica, simulando cenários técnicos exigidos pelo mercado de desenvolvimento de software. O trabalho é dividido em três projetos principais: Árvores de Busca, Sistemas de Busca e Benchmark de Ordenação.

2. PROJETO 1: Árvores e Balanceamento

Nesta etapa, foram implementadas as operações de inserção, remoção, busca e cálculo de altura para três tipos de árvores:

Árvore Binária de Busca (BST): Estrutura base, onde inserções sequenciais resultaram no pior caso de balanceamento, formando uma estrutura semelhante a uma lista encadeada.

Árvore AVL: Estrutura com autobalanceamento rigoroso por altura. Através das rotações, o fator de equilíbrio foi mantido, resultando em uma altura drasticamente menor em comparação à BST.

Árvore Rubro-Negra: Estrutura balanceada através de propriedades de cores (Vermelho/Preto). A implementação demonstrou eficiência superior para inserções e remoções frequentes.

Heurística do Caixeiro-Viajante:

Para resolver o problema do Caixeiro-Viajante em tempo hábil, foi implementada a heurística aproximada do Vizinho Mais Próximo. Os testes foram realizados com grafos de 100, 1000 e 1500 vértices, executados 5 vezes cada.

==========

Tamanho do grafo: 100

==========

Execução 1 | Tempo: 0.1589 ms | Custo: 566.0

Execução 2 | Tempo: 0.1581 ms | Custo: 440.0

Execução 3 | Tempo: 0.1662 ms | Custo: 441.0

Execução 4 | Tempo: 0.1767 ms | Custo: 542.0

Execução 5 | Tempo: 0.1931 ms | Custo: 612.0

MÉDIA:

Tempo médio: 0.1706 ms

Custo médio: 520.2

==========

Tamanho do grafo: 1000

==========

Execução 1 | Tempo: 5.8074 ms | Custo: 1308.0

Execução 2 | Tempo: 3.3097 ms | Custo: 1459.0

Execução 3 | Tempo: 1.5189 ms | Custo: 1588.0

Execução 4 | Tempo: 1.067 ms | Custo: 1508.0

Execução 5 | Tempo: 1.6638 ms | Custo: 1428.0

MÉDIA:

Tempo médio: 2.67336 ms

Custo médio: 1458.2

==========

Tamanho do grafo: 1500

==========

Execução 1 | Tempo: 2.3208 ms | Custo: 2098.0

Execução 2 | Tempo: 2.1267 ms | Custo: 1844.0

Execução 3 | Tempo: 2.0875 ms | Custo: 2014.0

Execução 4 | Tempo: 3.0864 ms | Custo: 1866.0

Execução 5 | Tempo: 3.6173 ms | Custo: 1972.0

MÉDIA:

Tempo médio: 2.6477399999999998 ms

Custo médio: 1958.8

3. Projeto 2: Sistemas de Busca

O projeto implementa três tipos de algoritmos de busca, cada um com características e desempenhos diferentes:

1. Busca Sequencial

A busca sequencial percorre o vetor **elemento por elemento**, do início até encontrar o valor desejado ou chegar ao final.

- **Funcionamento:** comparação direta em cada posição
- **Estrutura:** vetor não precisa estar ordenado
- **Complexidade:**
  - Melhor caso: O(1)
  - Pior caso: O(n)

É simples, porém **ineficiente para grandes volumes de dados**

2. Busca Binária

A busca binária funciona dividindo o vetor ao meio a cada passo, eliminando metade dos elementos da busca.

- **Funcionamento:** compara com o elemento do meio
- **Pré-requisito:** vetor **ordenado**
- **Complexidade:**
  - Melhor caso: O(1)
  - Pior caso: O(log n)

Muito mais eficiente que a sequencial, especialmente para grandes conjuntos

3. Busca em Árvore Binária de Busca (BST)

A busca em BST utiliza uma estrutura hierárquica onde valores menores ficam à esquerda e maiores à direita.

- **Funcionamento:** percorre a árvore seguindo a regra de ordenação
- **Estrutura:** árvore binária de busca
- **Complexidade:**
  - Caso médio: O(log n)
  - Pior caso: O(n) (árvore desbalanceada)

Mais eficiente que a sequencial, mas pode perder desempenho se a árvore não estiver balanceada

Resumo geral

- **Busca Sequencial:** simples, porém lenta
- **Busca Binária:** rápida, exige ordenação
- **Busca em BST:** eficiente, depende do balanceamento da árvore

Análise Estatística

A análise estatística foi desenvolvida com base nos tamanhos de entrada utilizados no projeto, além de outros valores menores empregados em testes iniciais. Essa variedade permite observar o comportamento dos algoritmos de busca em diferentes escalas de dados.

A análise considera o desempenho teórico dos algoritmos implementados — busca sequencial, busca binária e busca em árvore binária de busca (BST) — relacionando-os ao crescimento do volume de entrada.

Comportamento dos algoritmos

Busca Sequencial:
Apresenta crescimento linear (O(n)), ou seja, o tempo de execução aumenta proporcionalmente ao número de elementos. Nos maiores tamanhos de entrada, como 50000 elementos, o custo computacional tende a ser significativamente mais elevado, além de apresentar maior variação devido ao número de comparações realizadas.

Busca Binária:
Possui complexidade logarítmica (O(log n)), sendo altamente eficiente mesmo para grandes volumes de dados. O aumento do tamanho da entrada impacta pouco no tempo de execução, tornando esse algoritmo estável e previsível.

Busca em Árvore Binária de Busca (BST):
Apresenta comportamento médio O(log n), porém pode degradar para O(n) em casos de desbalanceamento. Assim, seu desempenho depende diretamente da forma como os dados são inseridos na árvore. Em cenários favoráveis, seu desempenho se aproxima da busca binária.

Comparação entre algoritmos

A comparação entre os algoritmos evidencia que:

- Para conjuntos menores, as diferenças de desempenho são pouco perceptíveis;

- Para entradas maiores (como 10000 e 50000 elementos), a busca binária e a BST apresentam desempenho superior à busca sequencial;
- A busca binária se destaca como a mais eficiente e estável entre as abordagens analisadas.

Interpretação dos resultados

Os resultados observados estão de acordo com a teoria da complexidade assintótica:

- Algoritmos O(n) apresentam crescimento linear e menor eficiência em grandes volumes de dados;
- Algoritmos O(log n) apresentam crescimento reduzido, sendo mais adequados para aplicações com grandes conjuntos de dados.

Além disso, fatores como a organização dos dados e o uso de estruturas adequadas influenciam diretamente o desempenho dos algoritmos.

Conclusão da análise estatística

Conclui-se que algoritmos mais eficientes, como a busca binária, oferecem melhor desempenho e maior estabilidade à medida que o volume de dados aumenta. A busca sequencial, embora simples, torna-se inviável para grandes conjuntos de dados. Já a busca em árvore binária de busca apresenta desempenho intermediário, podendo ser eficiente quando a estrutura se mantém balanceada.

4. Projeto 3: Benchmark de Ordenação:

Para este projeto, foram selecionados os algoritmos **Bubble Sort** e **Insertion Sort.**

Bubble Sort: Ordena um vetor comparando elementos adjacentes e trocando-os de posição quando estão fora de ordem. Repete o processo até que o vetor esteja completamente ordenado. Simples, mas ineficiente para vetores grandes — complexidade $O(n^2)$ no caso médio/pior.

Insertion Sort: Ordena o vetor inserindo cada elemento na posição correta dentro da parte já ordenada, deslocando os elementos maiores para a direita. Funciona bem para vetores pequenos ou quase ordenados — complexidade O(n) no melhor caso e $O(n^2)$ no pior.

Resultados do Benchmark:

==========

Caso: MELHOR

Tamanho: 1000

Bubble Sort:

Média (ns): 197626.0

Desvio padrão (ns): 349283.97662074334

Teoria: Melhor O(n), Médio/Pior O(n²)

Insertion Sort:

Média (ns): 23256.0

Desvio padrão (ns): 15839.566408207013

Teoria: Melhor O(n), Médio/Pior O(n²)

==========

Caso: MELHOR

Tamanho: 3000

Bubble Sort:

Média (ns): 885185.0

Desvio padrão (ns): 135436.72498624588

Teoria: Melhor O(n), Médio/Pior O(n²)

Insertion Sort:

Média (ns): 3495.0

Desvio padrão (ns): 11156.535080391222

Teoria: Melhor O(n), Médio/Pior O(n²)

==========

Caso: MELHOR

Tamanho: 5000

Bubble Sort:

Média (ns): 2355185.0

Desvio padrão (ns): 262495.9235397761

Teoria: Melhor O(n), Médio/Pior O(n²)

Insertion Sort:

Média (ns): 1749.0

Desvio padrão (ns): 1043.5990609424675

Teoria: Melhor O(n), Médio/Pior O(n²)

==========

Caso: MEDIO

Tamanho: 1000

Bubble Sort:

Média (ns): 687309.0

Desvio padrão (ns): 506867.20511293685

Teoria: Melhor O(n), Médio/Pior O(n²)

Insertion Sort:

Média (ns): 100938.0

Desvio padrão (ns): 284760.16813451983

Teoria: Melhor O(n), Médio/Pior O(n²)

==========

Caso: MEDIO

Tamanho: 3000

Bubble Sort:

Média (ns): 4612475.0

Desvio padrão (ns): 321317.5159791324

Teoria: Melhor O(n), Médio/Pior O(n²)

Insertion Sort:

Média (ns): 433638.0

Desvio padrão (ns): 47046.01955532476

Teoria: Melhor O(n), Médio/Pior O(n²)

==========

Caso: MEDIO

Tamanho: 5000

Bubble Sort:

Média (ns): 1.4304369E7

Desvio padrão (ns): 4337623.865083164

Teoria: Melhor O(n), Médio/Pior O(n²)

Insertion Sort:

Média (ns): 1155566.0

Desvio padrão (ns): 196854.0729677697

Teoria: Melhor O(n), Médio/Pior O(n²)

==========

Caso: PIOR

Tamanho: 1000

Bubble Sort:

Média (ns): 668035.0

Desvio padrão (ns): 50188.69070816652

Teoria: Melhor O(n), Médio/Pior O(n²)

Insertion Sort:

Média (ns): 118336.0

Desvio padrão (ns): 49748.26533659239

Teoria: Melhor O(n), Médio/Pior O(n²)

==========

Caso: PIOR

Tamanho: 3000

Bubble Sort:

Média (ns): 5784912.0

Desvio padrão (ns): 221852.58631803235

Teoria: Melhor O(n), Médio/Pior O(n²)

Insertion Sort:

Média (ns): 847442.0

Desvio padrão (ns): 183773.89704743162

Teoria: Melhor O(n), Médio/Pior O(n²)

==========

Caso: PIOR

Tamanho: 5000

Bubble Sort:

Média (ns): 1.5769324E7

Desvio padrão (ns): 523983.5322068815

Teoria: Melhor O(n), Médio/Pior O(n^2)

Insertion Sort:

Média (ns): 2176032.0

Desvio padrão (ns): 237561.3638115424

Teoria: Melhor O(n), Médio/Pior O(n^2)

Análise Assintótica

A análise assintótica tem como objetivo avaliar o comportamento dos algoritmos à medida que o tamanho da entrada cresce, independentemente de fatores específicos de hardware ou implementação. Essa abordagem permite compreender a eficiência teórica das soluções propostas, utilizando notações como Big-O (O), que descrevem o limite superior do tempo de execução.

No caso das árvores binárias de busca (BST), a complexidade das operações depende diretamente da altura da árvore. Em um cenário ideal (árvore balanceada), as operações possuem complexidade O(log n). No entanto, em situações de desbalanceamento — como inserções sequenciais — a estrutura pode degenerar para uma lista encadeada, resultando em complexidade O(n).

As árvores AVL e Rubro-Negra foram projetadas para manter o balanceamento da estrutura, garantindo altura proporcional a O(log n). Dessa forma, operações como inserção, remoção e busca mantêm desempenho eficiente mesmo para grandes volumes de dados. A AVL apresenta um balanceamento mais rigoroso, enquanto a Rubro-Negra oferece maior eficiência em operações dinâmicas devido ao menor número de rotações.

Nos algoritmos de busca, a busca sequencial apresenta complexidade O(n), pois pode ser necessário percorrer todos os elementos. Já a busca binária possui complexidade O(log n), sendo significativamente mais eficiente, porém exige que os dados estejam previamente ordenados. A busca em BST apresenta comportamento semelhante à busca binária em casos médios, mas pode degradar para O(n) se a árvore estiver desbalanceada.

Para os algoritmos de ordenação, tanto o Bubble Sort quanto o Insertion Sort apresentam complexidade O(n^2) nos casos médio e pior. No entanto, ambos podem atingir O(n) no melhor caso, quando os dados já estão ordenados ou quase ordenados. Apesar da simplicidade, esses algoritmos não são recomendados para grandes volumes de dados devido ao crescimento quadrático do tempo de execução.

Por fim, na heurística do Caixeiro-Viajante (Vizinho Mais Próximo), a complexidade é aproximadamente $O(n^2)$, considerando que, para cada vértice, é necessário avaliar os vizinhos não visitados. Embora não seja uma solução ótima, apresenta desempenho viável para grandes entradas quando comparada a algoritmos exatos de complexidade exponencial.

De modo geral, a análise assintótica confirma que algoritmos com complexidade logarítmica ou linear são mais adequados para aplicações que lidam com grandes volumes de dados, enquanto algoritmos quadráticos tendem a se tornar inviáveis conforme o crescimento da entrada.

Metodologia Experimental

A metodologia experimental adotada neste trabalho teve como objetivo avaliar, de forma prática, o desempenho das estruturas de dados e algoritmos implementados, considerando diferentes cenários e volumes de entrada.

Os experimentos foram realizados em ambiente controlado, utilizando a mesma máquina e condições de execução, a fim de reduzir interferências externas nos resultados. Para cada algoritmo, foram definidos conjuntos de dados com tamanhos variados, permitindo observar o comportamento conforme o crescimento da entrada.

No caso das árvores (BST, AVL e Rubro-Negra), foram testadas operações fundamentais como inserção, remoção, busca e cálculo de altura. Os dados foram inseridos de forma sequencial para evidenciar o pior caso da BST e comparar com as estruturas balanceadas.

Para o problema do Caixeiro-Viajante, utilizou-se a heurística do Vizinho Mais Próximo. Os testes foram realizados com grafos de tamanhos 100, 1000 e 1500 vértices, com 5 execuções para cada tamanho. Foram coletadas métricas de tempo de execução e custo do caminho encontrado, sendo posteriormente calculadas as médias.

Nos algoritmos de busca, foram considerados diferentes tamanhos de vetores (1000, 5000, 10000 e 50000 elementos). Cada algoritmo foi executado sob as mesmas condições, permitindo comparação direta entre busca sequencial, binária e em árvore.

Para o benchmark de ordenação, os algoritmos Bubble Sort e Insertion Sort foram testados em três cenários: melhor, médio e pior caso. Os testes foram realizados com vetores de tamanhos 1000, 3000 e 5000, com múltiplas execuções para cálculo de média e desvio padrão, garantindo maior confiabilidade estatística.

Os dados coletados foram organizados e analisados com base em métricas de desempenho (tempo de execução) e comparados com as complexidades teóricas esperadas, permitindo validar empiricamente os resultados.

DISCUSSÃO

Os resultados obtidos ao longo dos experimentos demonstram, de forma consistente, a relação entre a complexidade teórica dos algoritmos e seu desempenho prático.

No contexto das árvores, observou-se que a BST apresentou degradação significativa quando submetida a inserções sequenciais, confirmando seu pior caso O(n). Em contrapartida, as árvores AVL e Rubro-Negra mantiveram alturas reduzidas, evidenciando a eficiência dos mecanismos de balanceamento. Isso reforça a importância do uso de estruturas balanceadas em aplicações que exigem desempenho previsível.

A heurística do Caixeiro-Viajante mostrou-se eficiente em termos de tempo de execução, mesmo para grafos maiores. No entanto, a variação nos custos das soluções indica que, apesar de rápida, a abordagem não garante a solução ótima, o que é esperado em algoritmos heurísticos. Ainda assim, seu uso é justificável em cenários onde o tempo é fator crítico.

Nos algoritmos de busca, os resultados confirmaram a superioridade da busca binária em grandes volumes de dados, devido à sua complexidade logarítmica. A busca sequencial, embora simples, apresentou crescimento linear significativo, tornando-se inadequada para grandes conjuntos. A busca em BST mostrou desempenho intermediário, dependente do balanceamento da estrutura, o que evidencia uma limitação importante dessa abordagem.

No benchmark de ordenação, o Bubble Sort apresentou desempenho significativamente inferior em todos os cenários, especialmente em casos médios e piores, devido à sua complexidade quadrática. O Insertion Sort, por sua vez, apresentou melhor desempenho, principalmente em casos favoráveis, confirmando sua eficiência para conjuntos pequenos ou parcialmente ordenados.

De forma geral, os resultados experimentais estão alinhados com a teoria da complexidade dos algoritmos, demonstrando que a escolha adequada da estrutura de dados e do algoritmo impacta diretamente a eficiência das aplicações. Além disso, evidencia-se que fatores como organização dos dados, tamanho da entrada e natureza do problema influenciam significativamente o desempenho observado.

CONCLUSÃO:

O relatório demonstrou, na prática, como diferentes estruturas de dados e algoritmos impactam diretamente o desempenho de sistemas. As árvores balanceadas (AVL e Rubro-Negra) mostraram maior eficiência em comparação à BST simples, enquanto a heurística do Caixeiro-Viajante evidenciou a utilidade de soluções aproximadas para problemas complexos.

Nos algoritmos de busca, confirmou-se que a busca binária é a mais estável e eficiente em grandes volumes de dados, seguida pela BST quando balanceada, e que a busca sequencial se torna inviável em cenários maiores.

Por fim, o benchmark de ordenação destacou a diferença entre algoritmos simples e pouco eficientes, como o Bubble Sort, e alternativas mais adequadas para casos específicos, como o Insertion Sort.

Em síntese, a escolha correta da estrutura e do algoritmo é essencial para garantir eficiência, escalabilidade e robustez em aplicações reais.